УТВЕРЖДЕН

RU.17701729.09.03.04 81 01-1-ЛУ

МОБИЛЬНОЕ ПРИЛОЖЕНИЕ ДЛЯ ГИМНАЗИИ ИМ. ПРИМАКОВА

Пояснительная записка

RU.17701729.09.03.04 81 01-1

Листов 33

2026

Инв.№ подл	Подп. и дата	Взам. инв.№	Инв.№ дубл.	Подп. и дата

АННОТАЦИЯ

Данный программный документ представляет собой пояснительную записку к программному проекту «Мобильное приложение для Гимназии им. Примакова» содержит следующие разделы: «Введение», «Назначение и область применения», «Технические характеристики», «Ожидаемые технико-экономические показатели», приложения.

Раздел «Введение» включает в себя наименование программы и документ, на основании которого ведётся разработка, с указанием организации, утвердившей данный документ.

В разделе «Назначение и область применения» содержатся функциональное и эксплуатационное назначение программы и краткая характеристика области её применения.

В разделе «Назначение разработки» указано функциональное и эксплуатационное назначение создаваемого программного продукта.

В разделе «Технические характеристики» присутствуют следующие подразделы: постановка задачи на разработку программы, описание функционирования программы, описание и обоснование алгоритма работы программы, описание и обоснование выбора метода организации входных и выходных данных, описание работы с базой данных, описание и обоснование выбора состава технических и программных средств.

Раздел «Требования к программным документам» содержит указание на предварительный состав программной документации и специальные требования к ней.

В разделе «Ожидаемые технико-экономические показатели» указана предполагаемая потребность и экономические преимущества разработки по сравнению с отечественными и зарубежными образцами или аналогами.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.102-77 [2]: Стадии разработки.
3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
4. ГОСТ 19.104-78 [4]: Основные надписи.
5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.
6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.404-79 [10]: Пояснительная записка. Требования к содержанию и оформлению.

Изменения к данной Пояснительной записке оформляются согласно ГОСТ 19.603-78 [12], ГОСТ 19.604-78 [13].

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ 5
1.1. Наименование программы 5
1.2. Документ(ы), на основании которого(ых) ведется разработка 5
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ 6
2.1. Функциональное назначение 6
2.2. Эксплуатационное назначение 6
2.3. Краткая характеристика области применения программы 6
3. ДИАГРАММЫ И МОДЕЛИ 8
3.1. Модель C4: Контекстная диаграмма (уровень 1) 8
3.2. Модель C4: Контейнерная диаграмма (уровень 2) 9
3.3. Модель C4: Компонентная диаграмма (уровень 3) 9
3.4. Диаграмма вариантов использования (Use Case) 11
3.5. ER-диаграмма базы данных 11
3.6. Диаграмма потока данных 13
3.7. Скриншоты приложения 14
3.7.1. Аутентификация и главный экран 15
3.7.2. Расписание и детали урока 15
3.7.3. Управление дедлайнами 16
3.7.4. Stories и объявления 16
3.7.5. Рейтинг учителей 17
3.7.6. Меню столовой 17
3.7.7. Дополнительные функции 18
3.7.8. Настройки и темная тема 18
4. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ 19
4.1. Постановка задачи на разработку программы 19
4.2. Описание и обоснование выбора состава технических и программных средств 19
4.2.1. Состав технических и программных средств 19
4.2.2. Обоснование выбора технических и программных средств 20
4.3. Описание и обоснование архитектуры программы 20
4.4. Особенности функционала 22
4.4.1. Модуль аутентификации 22
4.4.2. Модуль расписания 23
4.4.3. Модуль Stories 23
4.4.4. Модуль рейтинга учителей 23
4.4.5. Модуль дедлайнов 23
4.4.6. Модуль редактирования профиля 23
4.4.7. Модуль меню столовой 23
4.4.8. Модуль навигации по кампусу 24
4.4.9. Модуль системы заявок 24
4.4.10. Основной сценарий работы 24

4.5. Обоснование выбора метода организации входных и выходных данных 24
5. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ 26
5.1. Ориентировочная экономическая эффективность 26
5.2. Предполагаемая потребность 26
5.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами 26
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ 28
ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ 30
ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ 31

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «Мобильное приложение для Гимназии им. Примакова».

Наименование программы на английском языке – «Mobile Application For Primakov School».

1.2. Документ(ы), на основании которого(ых) ведется разработка

Разработка ведется на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта.

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

2.1. Функциональное назначение

Мобильное приложение для Гимназии им. Примакова представляет собой кроссплатформенное мобильное приложение для автоматизации доступа к информации об учебном процессе в Гимназии им. Примакова. Основные функции включают:

- Централизованное управление расписанием занятий с указанием времени, кабинета и преподавателя
- Трекинг дедлайнов по домашним заданиям с возможностью отметки о выполнении
- Модуль рейтинга учителей с анонимной оценкой и комментариями (шкала 1-5)
- Систему Stories с фотографиями и гиперссылками на новости гимназии
- Редактирование профиля пользователя (имя, фамилия, смена пароля)
- Просмотр меню столовой с информацией о блюдах (калорийность, БЖУ, аллергены)
- Навигацию по кампусу (поиск аудиторий по номеру, корпусу, этажу)
- Систему заявок для сообщения о проблемах (IT, техническое обслуживание, уборка)
- Просмотр объявлений гимназии

2.2. Эксплуатационное назначение

Программа предназначена для ежедневного использования учащимися, преподавателями и обслуживающим персоналом Гимназии им. Примакова. Приложение оптимизировано для работы на мобильных устройствах под управлением iOS и Android и требует постоянного интернет-соединения для работы.

2.3. Краткая характеристика области применения программы

Разработка относится к области образовательных технологий и предназначена для автоматизации следующих процессов гимназии:

- Информирование о расписании занятий и изменениях
- Организация учебного процесса через систему дедлайнов
- Анонимная система рейтингов преподавателей для обратной связи
- Информирование о новостях и событиях через Stories с фотографиями
- Просмотр меню столовой с детальной информацией о блюдах
- Навигация по кампусу и система заявок
- Просмотр объявлений гимназии

Целевая аудитория — учащиеся, преподаватели и обслуживающий персонал гимназии. Программа разработана в рамках курсового проекта и интегрируется с backend через REST API.

Приложение не стремится повторить функционал государственных решений (МЭШ, Дневник.ру), а пересматривает привычную логику работы с образовательными цифровыми сервисами: фокус на уникальности гимназии, гибкости, интеграции задач, оперативной обратной связи и создании комфортной образовательной среды. Приложение адаптируется к потребностям гимназии с индивидуальным функционалом, фирменным дизайном и специальными сервисами. Всё в одном месте: расписание, домашние задания, уведомления, важные новости и объявления доступны через

одну платформу, что снижает нагрузку на администрацию и улучшает коммуникацию между всеми участниками образовательного процесса.

3. ДИАГРАММЫ И МОДЕЛИ

3.1. Модель C4: Контекстная диаграмма (уровень 1)

Контекстная диаграмма показывает систему Мобильное приложение для Гимназии им. Примакова в окружении внешних пользователей и систем. Пользователь взаимодействует с мобильным приложением, которое отправляет HTTP запросы к Backend API. Приложение может открывать внешние ссылки через браузер для доступа к полным версиям новостей на сайте гимназии primakov.school.

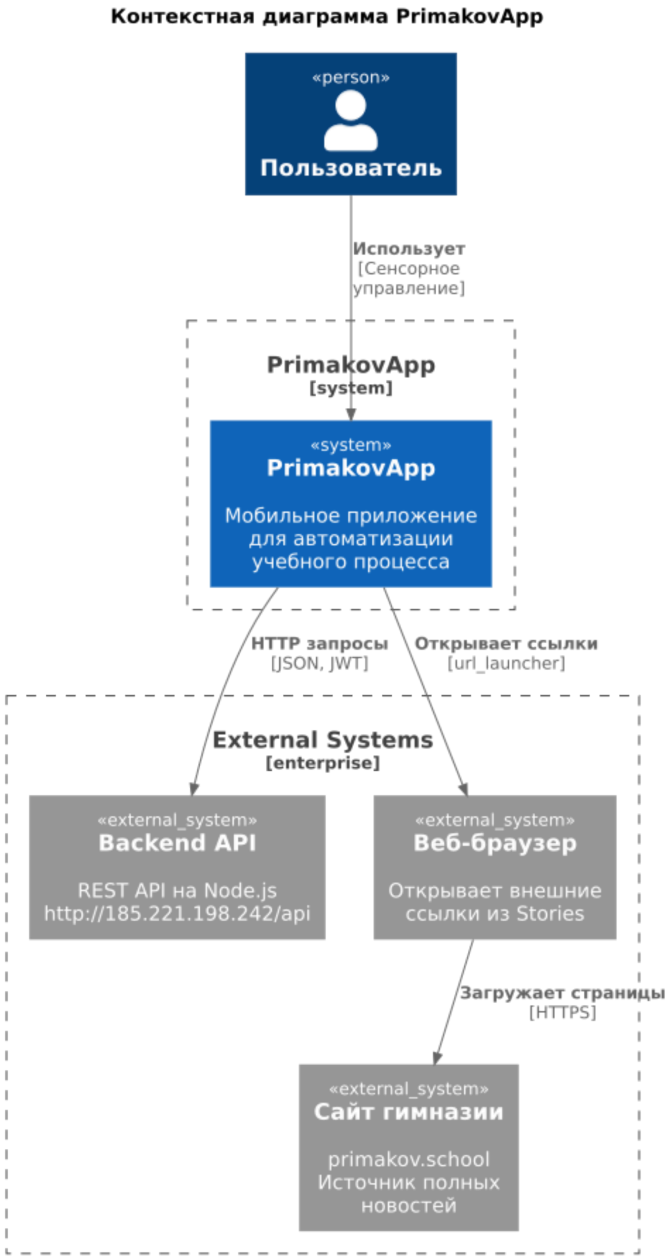


Рис. 1. Контекстная диаграмма системы PrimakovApp

3.2. Модель C4: Контейнерная диаграмма (уровень 2)

Контейнерная диаграмма показывает основные технологические компоненты системы и их взаимодействие. Мобильное приложение состоит из Flutter App (UI), Riverpod (управление состоянием), Dio (HTTP клиент) и shared_preferences (локальное хранилище для JWT токенов и настроек UI). Backend сервер включает REST API на Node.js и Express.js, Auth Service с JWT токенами (7 дней действия) и бизнес-логику на TypeScript. База данных PostgreSQL 14+ содержит 13 таблиц и развернута нативно на VPS.

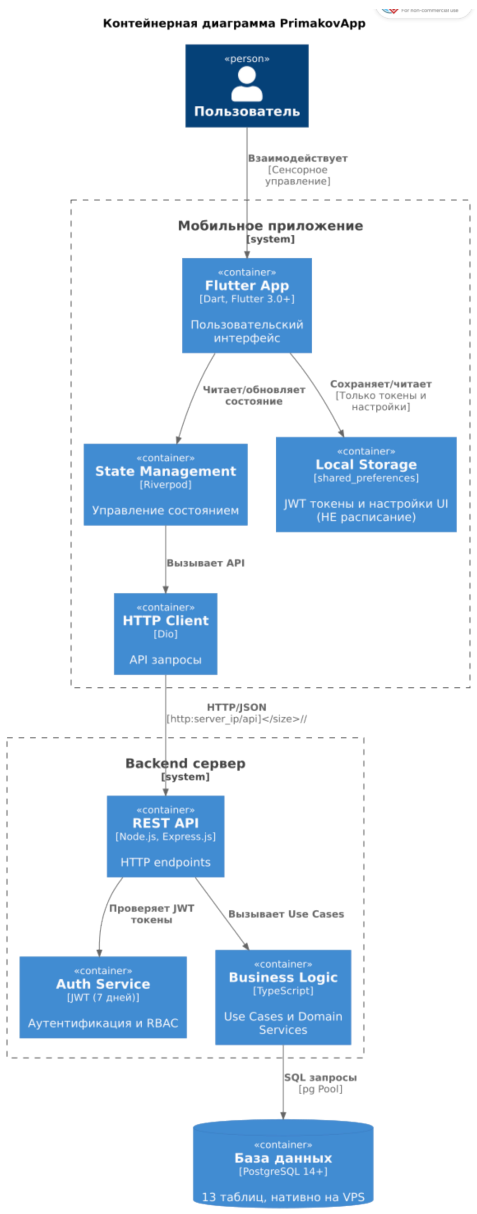


Рис. 2. Контейнерная диаграмма системы PrimakovApp

3.3. Модель C4: Компонентная диаграмма (уровень 3)

Компонентная диаграмма показывает внутреннее устройство backend сервера. Вся логика разделена на четыре слоя:

Первый слой (Presentation) — отвечает за приём запросов. Здесь находятся контроллеры, которые обрабатывают URL, и middleware, которые проверяют авторизацию и права доступа.

Второй слой (Application) — содержит сценарии использования (Use Cases). Каждая операция, например «получить расписание» или «создать дедлайн», вынесена в отдельный класс.

Третий слой (Domain) — содержит бизнес-сущности (пользователь, расписание, дедлайн) и объекты-значения (email, пароль, номер аудитории). Здесь описаны правила, как эти объекты могут себя вести.

Четвёртый слой (Infrastructure) — отвечает за хранение данных. Репозитории работают с базой данных PostgreSQL через специальный пул соединений (pg Pool). Всего реализовано 9 таких репозиториев.

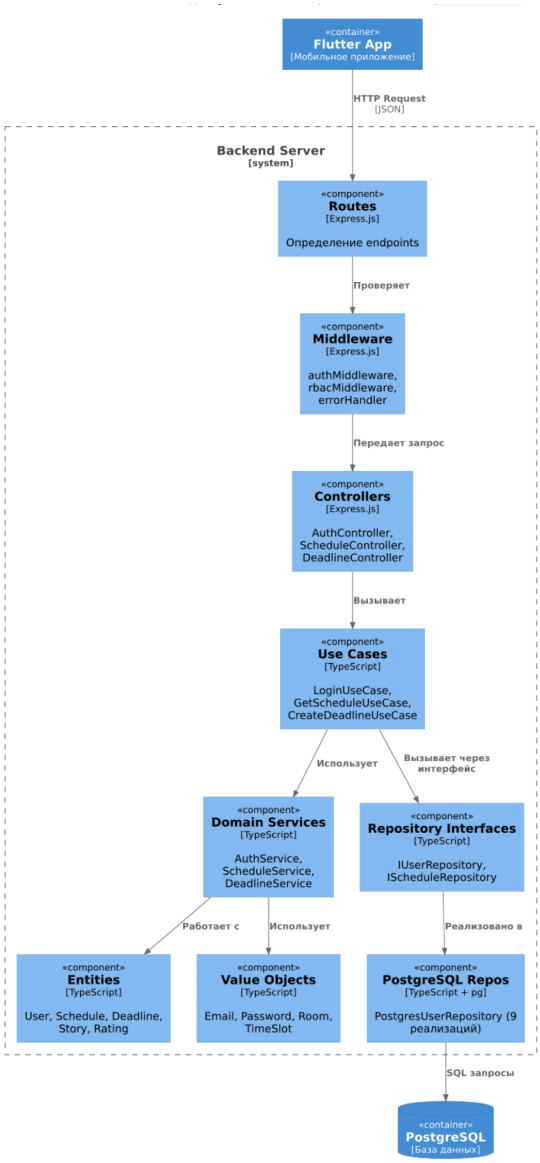


Рис. 3. Компонентная диаграмма Backend сервера

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03.04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.4. Диаграмма вариантов использования (Use Case)

Все пользователи работают с одинаковым интерфейсом, но доступ к функциям ограничивается на стороне сервера в зависимости от роли. Основные возможности системы:

- вход в систему с использованием токенов (срок действия — 7 дней);
- просмотр расписания занятий;
- управление заданиями с возможностью отметки о выполнении;
- просмотр историй с переходом по внешним ссылкам;
- оценка учителей по пятибалльной шкале;
- редактирование профиля и смена пароля;
- просмотр меню столовой;
- создание заявок.

Администраторы имеют дополнительные права: они могут просматривать данные об оценках учителей (включая информацию о том, кто поставил оценку). Все проверки прав доступа выполняются на стороне сервера с использованием специальных программных прослоек.

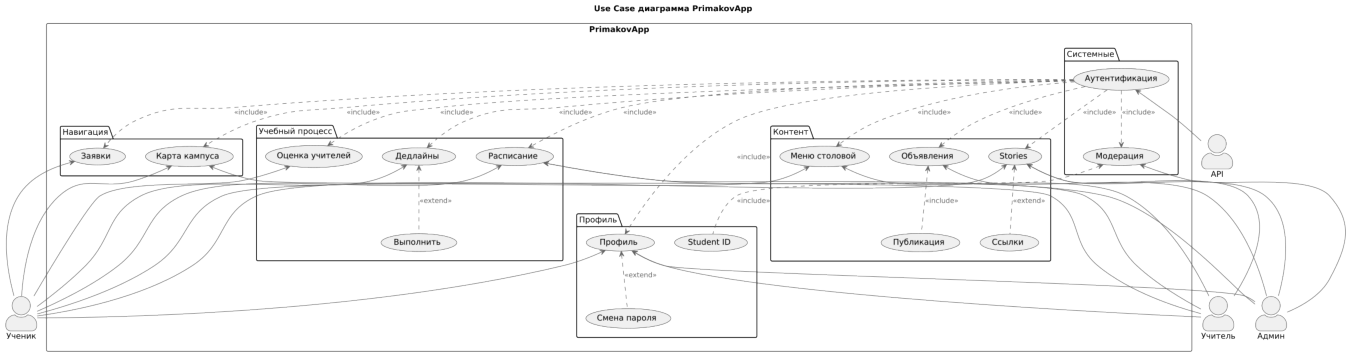


Рис. 4. Диаграмма вариантов использования

3.5. ER-диаграмма базы данных

Диаграмма сущность-связь показывает структуру базы данных PostgreSQL с 13 таблицами и их взаимосвязями. Центральная таблица users связана с schedules (расписания), deadlines (дедлайны), ratings (рейтинги учителей), stories (новости), announcements (объявления) и requests (заявки). Таблица schedules содержит lessons (уроки), которые связаны с rooms (аудитории). Stories имеют связь с story_views для отслеживания просмотров. Таблица canteen_menu содержит menu_items с информацией о блюдах (калорийность, БЖУ, аллергены). Все таблицы используют UUID как первичные ключи и содержат временные метки created_at и updated_at.

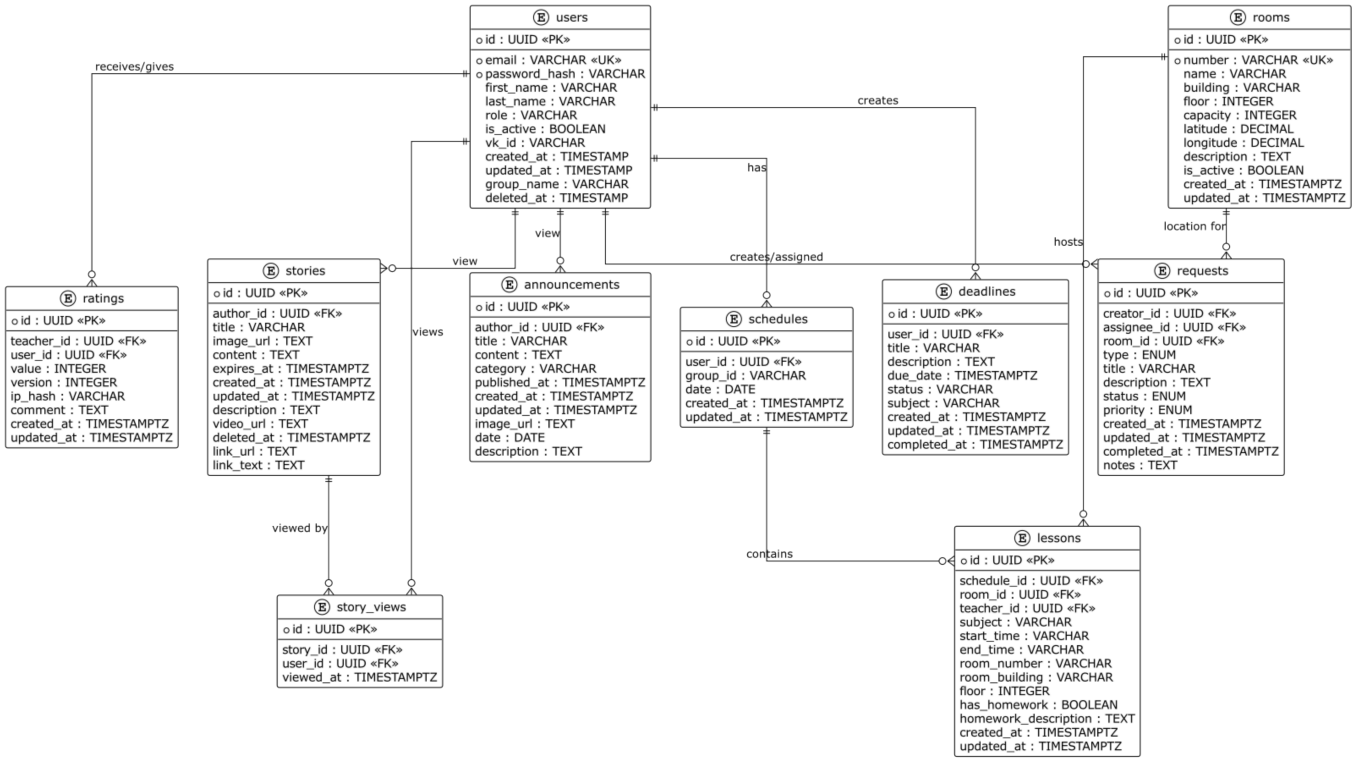


Рис. 5. ER-диаграмма базы данных

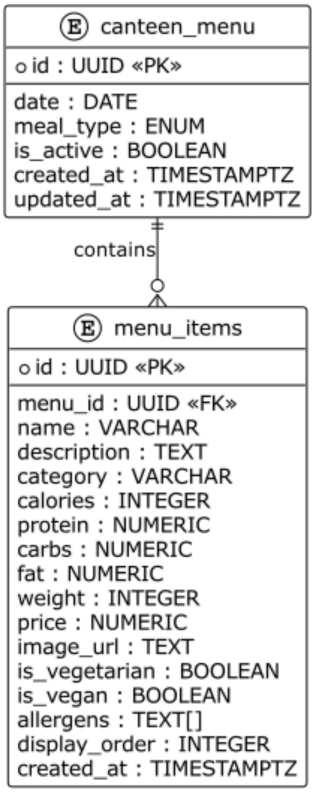


Рис. 6. ER-диаграмма: Меню столовой

3.6. Диаграмма потока данных

Диаграмма показывает путь данных от действия пользователя до обновления экрана через все слои приложения. Важно: расписание и другие данные не сохраняются на устройстве, каждый раз запрашиваются с сервера по протоколу HTTP. Движение данных начинается с действия пользователя (нажатие кнопки). Элемент управления вызывает способ управления состоянием приложения, который отправляет запрос по сети через службу связи с сервером, добавляя в заголовок запроса ключ доступа для подтверждения личности пользователя.

Серверная часть сначала проверяет этот ключ и права доступа, затем выполняет сценарий использования, который через посредника работы с данными обращается к базе данных PostgreSQL, используя пул соединений. Результат возвращается в виде специального контейнера «Результат» (успех или ошибка), преобразуется в ответ в формате JSON, разбирается в службе связи и передаётся обратно в способ управления состоянием. Способ управления состоянием обновляет внутренние данные (не сохраняя их на устройстве), снимает флаг ожидания загрузки и оповещает элемент управления. Элемент управления заново отрисовывает экран, и пользователь видит обновлённый интерфейс.

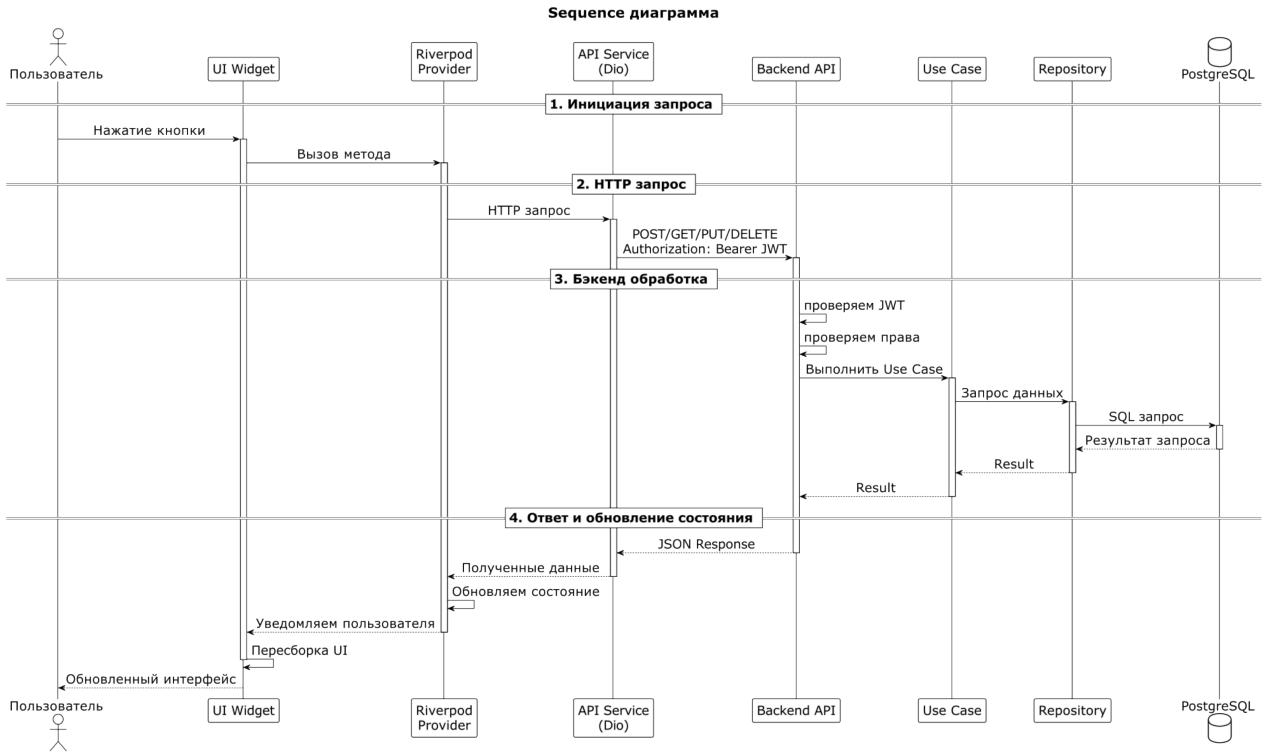


Рис. 7. Диаграмма потока данных

3.7. Скриншоты приложения

Ниже представлены скриншоты основных экранов мобильного приложения PrimakovApp.

3.7.1. Аутентификация и главный экран

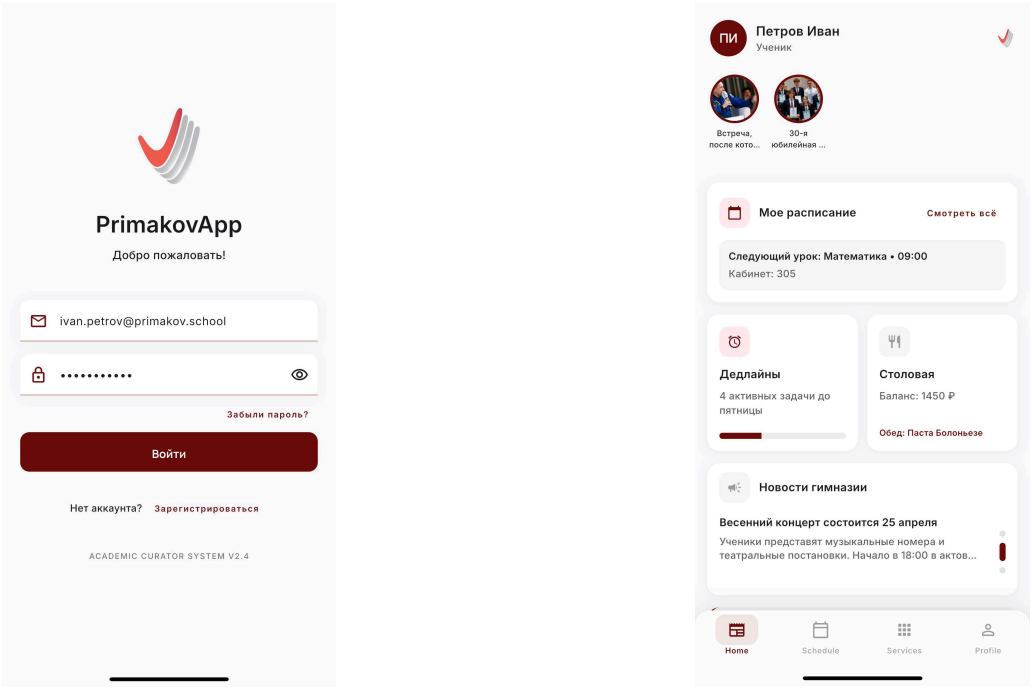


Рис. 8. Экран входа и главный экран приложения

3.7.2. Расписание и детали урока

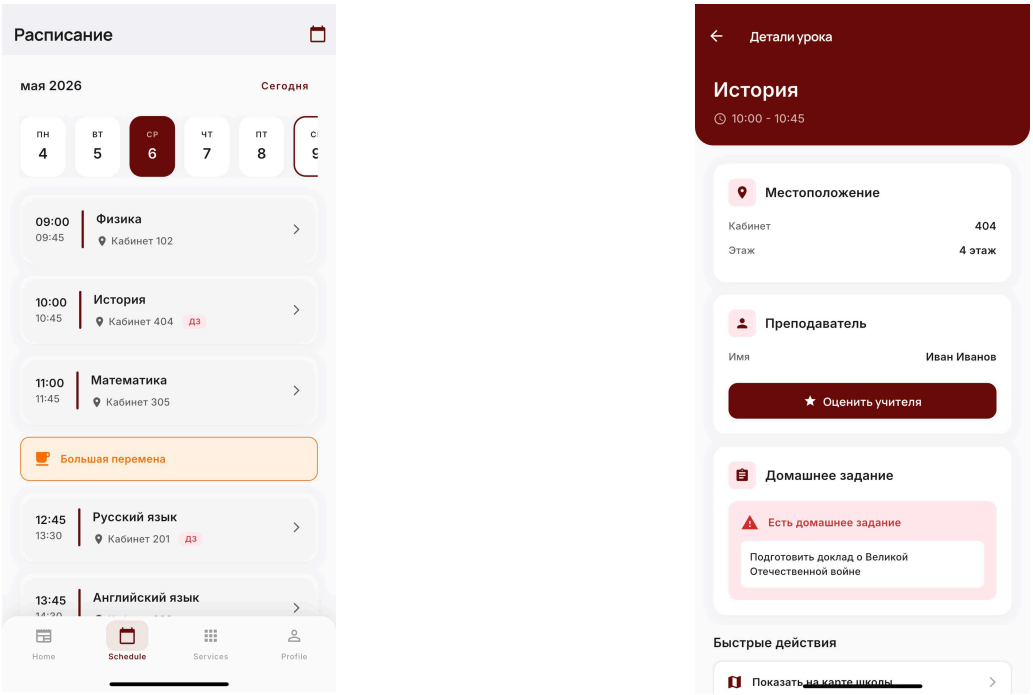


Рис. 9. Экран расписания и детали урока

3.7.3. Управление дедлайнами

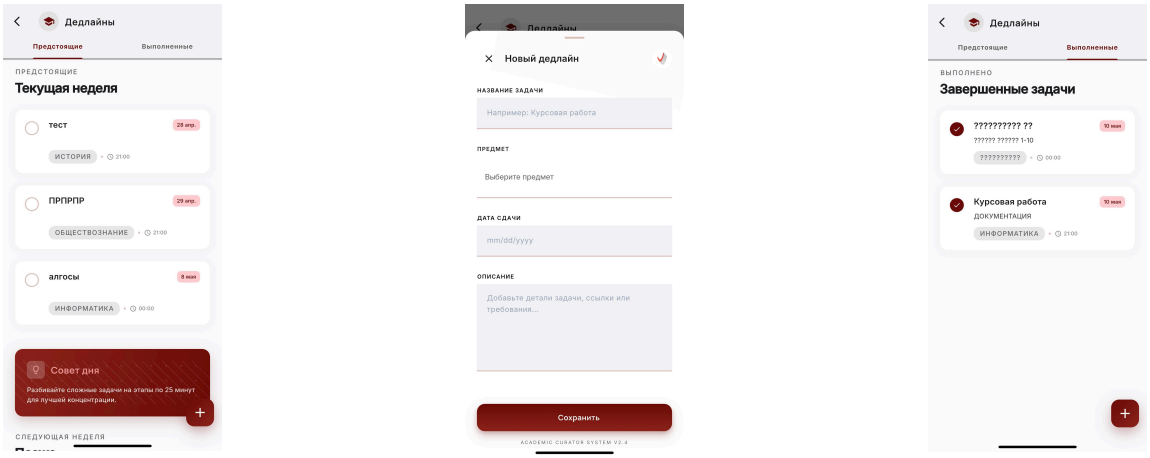


Рис. 10. Список дедлайнов, добавление и завершенные дедлайны

3.7.4. Stories и объявления

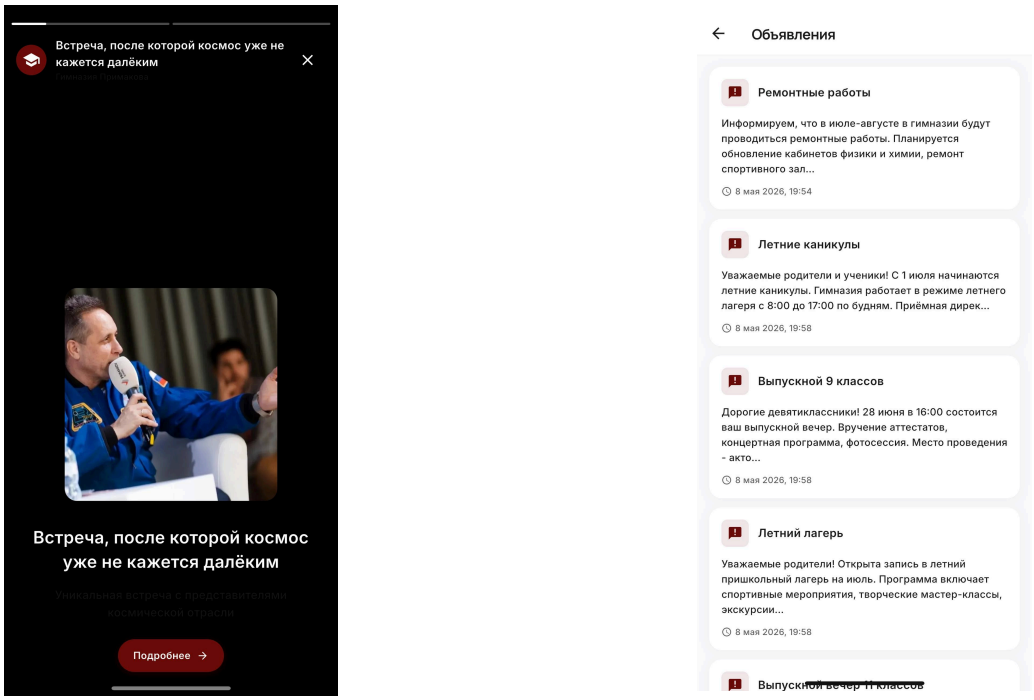


Рис. 11. Stories с фотографиями и экран объявлений

3.7.5. Рейтинг учителей

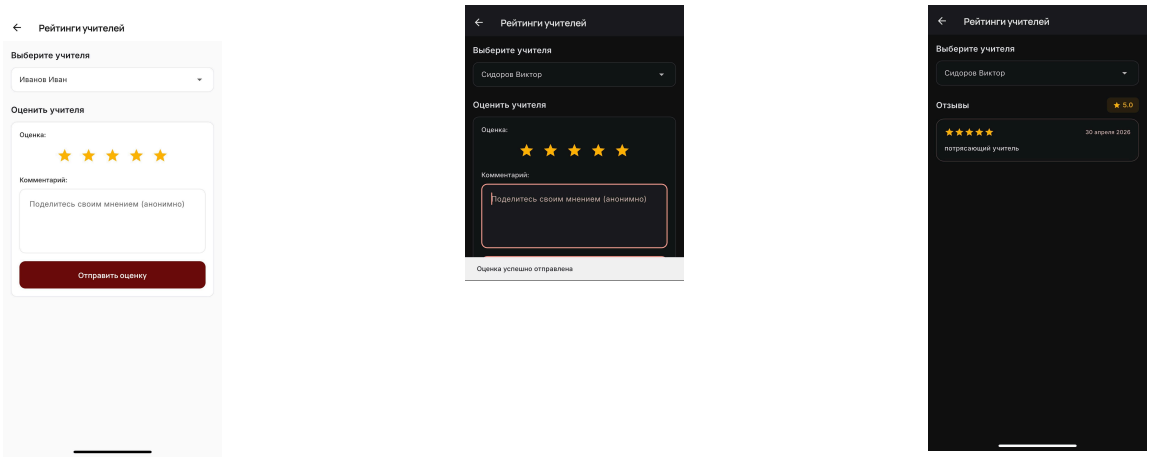


Рис. 12. Рейтинг учителей, успешная оценка и админ панель

3.7.6. Меню столовой

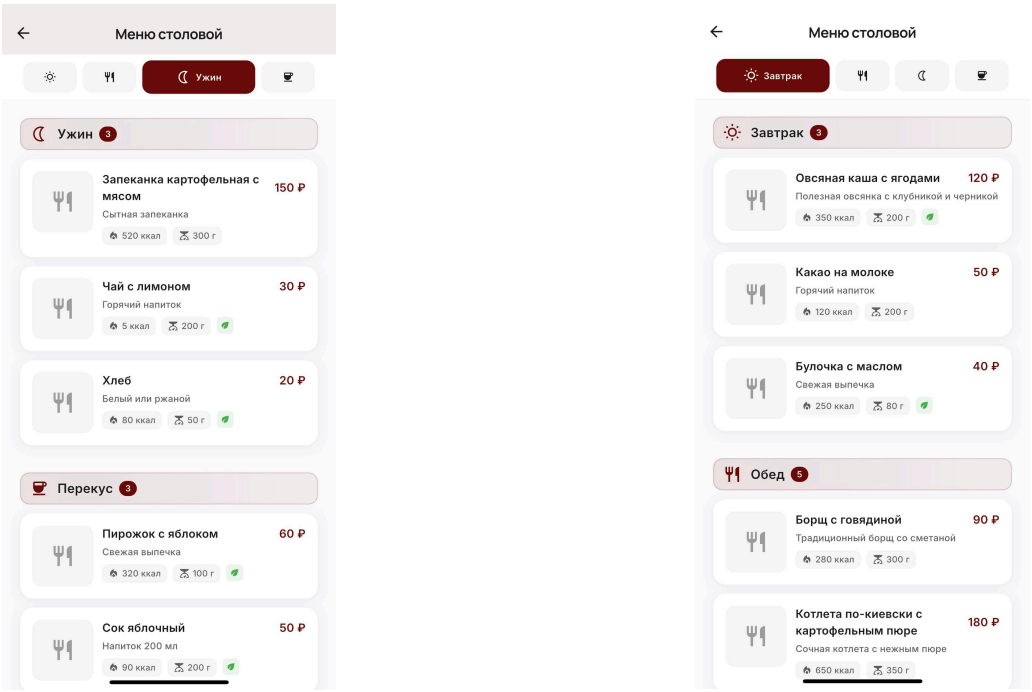


Рис. 13. Меню столовой с информацией о блюдах

3.7.7. Дополнительные функции

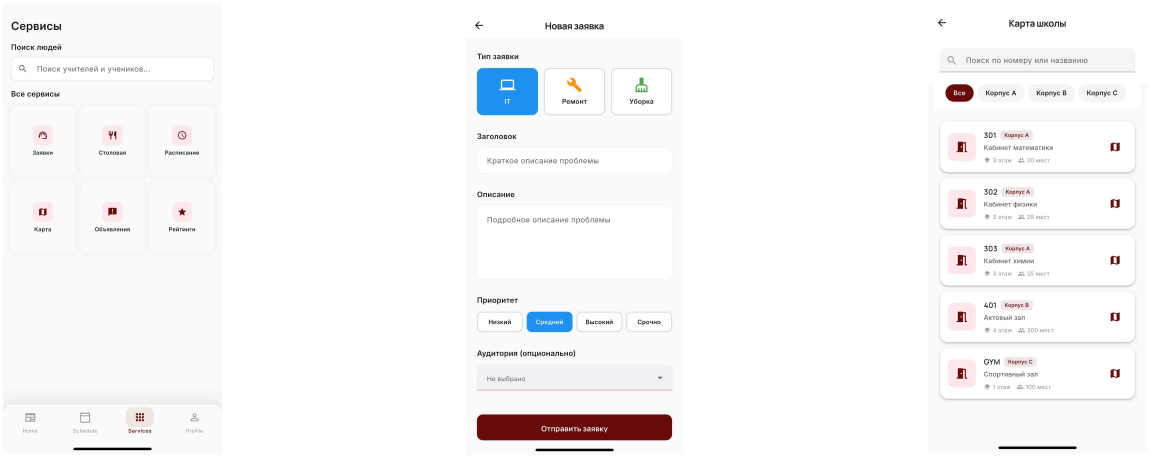


Рис. 14. Экран сервисов, создание заявки и навигация по кампусу

3.7.8. Настройки и темная тема

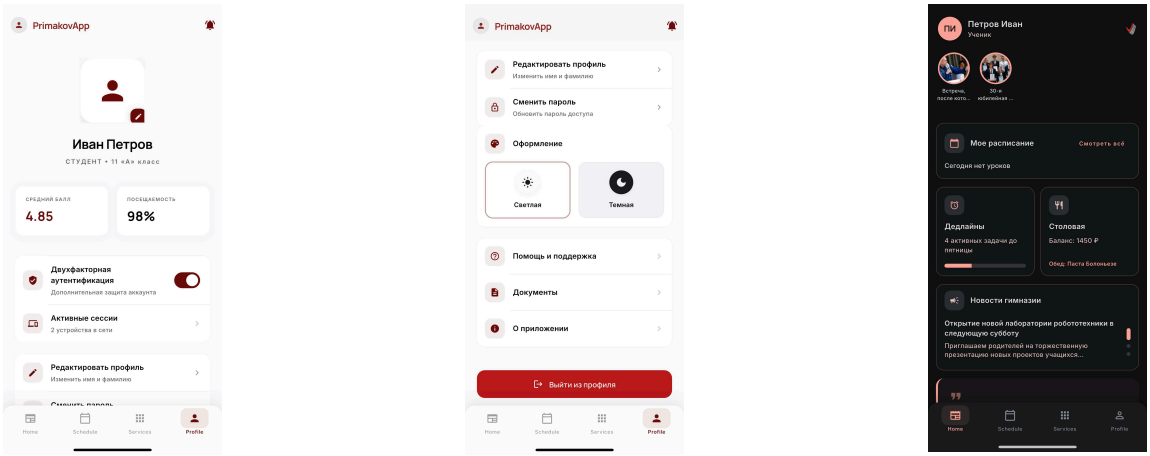


Рис. 15. Настройки приложения и пример темной темы

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03.04 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

4.1. Постановка задачи на разработку программы

Задача разработки — создание кроссплатформенного мобильного приложения PrimakovApp для автоматизации учебного процесса в Гимназии им. Примакова. Основные требования включают:

- Реализацию системы аутентификации с JWT токенами (срок действия 7 дней) и разграничением прав доступа;
- Организацию персонализированного доступа к расписанию занятий по идентификатору пользователя;
- Разработку модуля Stories для отображения школьных новостей с фотографиями и гиперссылками на внешние ресурсы;
- Создание системы рейтинга учителей с возможностью оставления анонимных отзывов по шкале от 1 до 5;
- Внедрение механизма отслеживания дедлайнов по учебным заданиям с фильтрацией по статусу;
- Реализацию редактирования профиля пользователя (имя, фамилия, смена пароля);
- Разработку модуля меню столовой с информацией о калорийности, БЖУ и аллергенах;
- Создание системы навигации по кампусу с поиском аудиторий;
- Внедрение системы заявок для сообщения о проблемах.

Технические ограничения:

- Поддержка платформ Android 10+ и iOS 14+;
- Работа требует постоянного интернет-соединения (нет офлайн-режима);
- Обеспечение времени отклика API не более 5 секунд при стабильном соединении;

4.2. Описание и обоснование выбора состава технических и программных средств

4.2.1. Состав технических и программных средств

Для разработки проекта используются следующие технологии:

Backend:

- язык программирования: TypeScript;
- веб-фреймворк: Express.js 4.18.2;
- система управления базами данных: PostgreSQL 14+;
- аутентификация: JWT (JSON Web Token) со сроком действия 7 дней;
- хеширование паролей: bcrypt.

Frontend:

- фреймворк: Flutter (язык Dart);
- управление состоянием: Riverpod 2.x;
- HTTP-клиент: Dio 5.x;
- локальное хранилище: shared_preferences (для токенов доступа и настроек интерфейса);
- открытие внешних ссылок: url_launcher 6.x;
- навигация между экранами: Navigator 2.0.

Инструменты разработки:

- Docker — для контейнеризации серверной части;
- Git — для контроля версий и отслеживания изменений в коде;
- GitHub — для удалённого хранения репозитория;
- VS Code — среда разработки для клиентской части на Flutter;
- Xcode — среда для сборки и установки iOS-версии приложения;
- Jest — для модульного тестирования серверной части;
- pgAdmin — для администрирования базы данных PostgreSQL.
- Visual Paradigm Online — для создания диаграмм;
- DeepSeek — для уточнения формулировок и сбора информации(ИИ-инструмент);
- Stitch AI — для разрабоки скелета пользовательского интерфейса (ИИ-инструмент).
- Figma - для разработки пользовательского интерфейса;
- Copilot CLI - для отлаживания кода и поиска ошибок (ИИ-инструмент);

4.2.2. Обоснование выбора технических и программных средств

Язык TypeScript выбран для разработки серверной части благодаря строгой системе типов, которая снижает количество ошибок на этапе написания кода и упрощает его дальнейшее сопровождение. Веб-фреймворк Express.js предоставляет минималистичную и гибкую основу для создания REST API с возможностью подключения промежуточных обработчиков запросов.

В качестве системы управления базами данных используется PostgreSQL. Этот выбор обусловлен надёжностью реляционных баз данных, поддержкой транзакций, механизма миграций и выполнения сложных запросов. База данных развёрнута непосредственно на виртуальном сервере, что обеспечивает стабильную работу приложения.

Клиентская часть разработана с использованием фреймворка Flutter, который позволяет создавать приложения для iOS и Android из единой кодовой базы с производительностью, близкой к нативной. Управление состоянием приложения реализовано с помощью библиотеки Riverpod, обеспечивающей безопасную работу с данными на этапе компиляции и минимальный объём вспомогательного кода. Для выполнения HTTP-запросов выбрана библиотека Dio, поддерживающая перехватчики запросов, которые автоматически добавляют токены доступа в заголовки.

Для открытия внешних веб-страниц (например, полных версий новостей из ленты «Истории») используется пакет url_launcher, который вызывает системный браузер устройства.

Локальное хранение данных организовано с помощью библиотеки shared_preferences. В ней сохраняются токены доступа и настройки пользовательского интерфейса (выбранная тема оформления, язык). Следует отметить, что учебные данные — расписание, дедлайны и другая информация — на устройстве не сохраняются. При каждом обращении приложение запрашивает актуальные сведения с сервера.

4.3. Описание и обоснование архитектуры программы

Система построена по принципам чистой архитектуры с чётким разделением ответственности между слоями. Серверная часть состоит из четырёх основных слоев:

1. **Доменный слой**:

- Сущности: User (пользователь), Schedule (расписание), Lesson (урок), Deadline (дедлайн), Story (история), Announcement (объявление), Rating (рейтинг)
- Объекты-значения: Email, Password, Room, TimeSlot — неизменяемые объекты со встроенной проверкой корректности данных
- Интерфейсы репозиториев: IUserRepository, IScheduleRepository, IDeadlineRepository — определяют способы работы с данными
- Доменные службы: AuthService, ScheduleService, DeadlineService, RatingService — обеспечивают бизнес-логику

2. **Слой приложения**:
   - Сценарии использования (Use Cases) — отдельные операции:
     - LoginUseCase, RegisterUseCase — аутентификация пользователя
     - GetScheduleByUserIdUseCase — получение расписания по идентификатору
     - CreateDeadlineUseCase, CompleteDeadlineUseCase — управление дедлайнами
     - RateTeacherUseCase, GetTeacherRatingsUseCase — система рейтингов
     - UpdateProfileUseCase, ChangePasswordUseCase — управление профилем
   - DTO (Data Transfer Objects) — объекты передачи данных для обмена информацией между слоями
3. **Слой инфраструктуры**:
   - Репозитории PostgreSQL — 9 реализованных компонентов (PostgresUserRepository, PostgresScheduleRepository и др.)
   - Конфигурация окружения, базы данных и JWT токенов
4. **Слой представления**:
   - Контроллеры — обработчики HTTP запросов (AuthController, ScheduleController, DeadlineController)
   - Маршруты — определение endpoints REST API
   - Middleware (промежуточные обработчики):
     - authMiddleware — проверка JWT токена
     - rbacMiddleware — проверка прав доступа
     - errorHandlerMiddleware — централизованная обработка ошибок
     - validationMiddleware — проверка корректности входных данных

Архитектурные паттерны:

- **Result Pattern** — явная обработка ошибок. Каждый Use Case возвращает объект Result с полями isSuccess, isFailure, value, error.
- **Repository Pattern** — абстракция доступа к данным через интерфейсы. Доменный слой зависит только от интерфейсов, а слой инфраструктуры предоставляет конкретные реализации для PostgreSQL.
- **Dependency Injection** — внедрение зависимостей через конструкторы, что обеспечивает гибкость и упрощает тестирование.
- **Single Responsibility** — каждый Use Case отвечает за одну операцию, каждый Value Object — за валидацию одного типа данных.

Архитектура клиентской части на Flutter:

Клиентское приложение построено на трёх основных слоях:

1. **Слой представления**:
   - Экраны (21 экран): LoginScreen, HomeScreen, ScheduleScreen, DeadlineScreen, ProfileScreen и другие
   - Переиспользуемые виджеты (более 10): LessonCard, DeadlineCard, StoryCard, CustomButton
   - Конфигурация оформления — тема, цветовая палитра, типографика
2. **Слой управления состоянием (Riverpod)**:
   - StateNotifier классы для управления состоянием (AuthNotifier, ScheduleNotifier, DeadlineNotifier)
   - State классы — неизменяемые объекты состояния с методом copyWith()
   - Провайдеры Riverpod — обеспечивают обновление интерфейса при изменении данных
3. **Слой служб (API интеграция)**:
   - API Services — HTTP клиенты на основе Dio (AuthService, ScheduleService, DeadlineService)
   - API Config — базовый URL, интерсепторы для автоматического добавления JWT токенов в заголовки
   - Модели данных — классы с методами fromJson() и toJson() для преобразования JSON

Поток данных в приложении:

Действие пользователя (нажатие кнопки) → Widget (элемент интерфейса) → Riverpod Provider (управление состоянием) → API Service (служба) → HTTP запрос через Dio → Backend API (серверная часть) → HTTP ответ с JSON → API Service парсит JSON → Provider обновляет состояние → Widget реагирует на изменение → UI обновляется

Преимущества архитектуры:

- Чёткое разделение ответственности между слоями
- Простота тестирования (возможность использования mock-объектов)
- Независимость бизнес-логики от внешних библиотек и фреймворков
- Возможность замены реализаций без изменения бизнес-логики
- Масштабируемость и удобство сопровождения кода

4.4. Особенности функционала

4.4.1. Модуль аутентификации

Обеспечивает регистрацию, вход и управление сессиями через JWT токены. Реализована система разграничения прав доступа (RBAC — Role-Based Access Control). Пароли хранятся в виде bcrypt-хешей. JWT токены имеют срок действия 7 дней и содержат userId, email и role пользователя. Токены автоматически добавляются к каждому HTTP запросу через Dio интерсепторы на стороне клиента.

4.4.2. Модуль расписания

Предоставляет персонализированное расписание для каждого пользователя по его userId. Данные запрашиваются с сервера при каждом обращении. API endpoint GET /api/schedule/user/:userId/:date возвращает список уроков в формате JSON с указанием предмета, времени начала и окончания, номера кабинета, этажа, преподавателя и статуса домашнего задания. Поддерживается фильтрация по датам через week selector в интерфейсе. Backend использует GetScheduleByUserIdUseCase для получения данных из PostgreSQL через PostgresScheduleRepository.

4.4.3. Модуль Stories

Отображает временные публикации новостей гимназии с фотографиями и гиперссылками. Каждая Story содержит заголовок, описание, изображение (280×280 пикселей), ссылку на полную новость. При клике на кнопку «Подробнее» открывается внешний браузер через пакет url_launcher. Система учитывает просмотры пользователей через таблицу story_views и отображает индикатор просмотренных историй. Stories автоматически переключаются с индикатором прогресса (progress bar).

4.4.4. Модуль рейтинга учителей

Позволяет оценивать преподавателей по шкале от 1 до 5 с обязательными комментариями (до 200 символов). Реализована защита от множественных оценок одного учителя одним пользователем через проверку в базе данных. Оценки полностью анонимны для обычных пользователей, но администраторы имеют доступ к детальной информации для модерации. API endpoint POST /api/ratings доступен авторизованным пользователям, GET /api/ratings?teacherId=X доступен администраторам.

4.4.5. Модуль дедлайнов

Пользователи могут создавать задания с указанием названия (до 100 символов), описания (до 500 символов), предмета и срока выполнения в формате ISO 8601. Система автоматически определяет статус задания: PENDING (ожидает выполнения), COMPLETED (выполнено), OVERDUE (просрочено). При отметке задания как выполненного сохраняется временная метка completedAt.

4.4.6. Модуль редактирования профиля

Пользователи могут редактировать имя и фамилию (максимум 50 символов каждое) через экран EditProfileScreen. Смена пароля осуществляется через отдельный экран ChangePasswordScreen с валидацией: минимум 8 символов, максимум 100 символов, проверка текущего пароля, запрет установки того же пароля. Backend endpoints: GET /api/users/me (получение профиля), PUT /api/users/me (обновление имени и фамилии), PUT /api/users/me/password (смена пароля). AuthState на клиенте автоматически синхронизируется после успешного обновления профиля.

4.4.7. Модуль меню столовой

Отображает меню на текущий день с разделением по типам приёма пищи: BREAKFAST (завтрак), LUNCH (обед), DINNER (ужин), SNACK (перекус). Для каждого блюда указаны: название,

калорийность (ккал), белки, жиры и углеводы (БЖУ в граммах), вес порции, цена и информация об аллергенах. Навигация между секциями реализована через горизонтальные вкладки с использованием пакета scrollable_positioned_list для надёжной прокрутки к любой секции. При открытии экрана автоматически прокручивается к секции, соответствующей времени суток. База данных содержит 90 записей меню на 5 дней.

4.4.8. Модуль навигации по кампусу

Позволяет искать аудитории по номеру, фильтровать по корпусу и этажу. Для каждой аудитории доступна информация о названии, вместимост. Данные хранятся в таблице rooms. Реализован через RoomService и RoomProvider с экраном RoomsListScreen.

4.4.9. Модуль системы заявок

Пользователи могут создавать заявки для сообщения о проблемах с выбором типа: IT (технические проблемы), MAINTENANCE (техническое обслуживание), CLEANING (уборка). Для каждой заявки указывается приоритет (LOW, MEDIUM, HIGH, URGENT), описание проблемы и привязка к конкретной аудитории через room_id. Статусы заявок: PENDING (ожидает), IN_PROGRESS (в работе), COMPLETED (выполнено), CANCELLED (отменено). Реализован через RequestService и CreateRequestScreen.

4.4.10. Основной сценарий работы

1. Пользователь входит в систему через экран LoginScreen, вводя email и пароль
2. Backend проверяет учётные данные через LoginUseCase и AuthService с использованием bcrypt
3. При успешной аутентификации генерируется JWT токен, который сохраняется в shared_preferences
4. Открывается главный экран (HomeScreen) с навигацией по 4 вкладкам: Главная, Расписание, Сервисы, Профиль
5. На главном экране отображаются виджеты: приветствие пользователя, Stories с фотографиями, ближайшие дедлайны (UpcomingDeadlinesWidget), расписание на сегодня.
6. При клике на Story открывается StoryViewerScreen с возможностью перехода по гиперссылке для просмотра новости на сайте гимназии
7. Пользователь может переключаться между вкладками через навигационную панель
8. Все запросы к API автоматически включают JWT токен
9. При истечении токена (срок действия 7 дней) или ошибке 401 Unauthorized пользователь перенаправляется на экран входа

4.5. Обоснование выбора метода организации входных и выходных данных

Входные данные включают:

1. Данные аутентификации:
- Email в формате user@primakov.school с валидацией
- Пароль длиной от 8 до 100 символов, хешируемый через bcrypt с 10 rounds
2. Запросы расписания:

- Идентификатор пользователя (UUID формат)
- Дата в формате YYYY-MM-DD (стандарт ISO 8601)

3. Данные дедлайнов:
- Название задачи (до 100 символов)
- Описание (до 500 символов)
- Дата выполнения в формате ISO 8601
- Идентификатор предмета (опционально)

4. Оценки преподавателей:
- Идентификатор преподавателя (UUID)
- Числовая оценка от 1 до 5
- Обязательный комментарий (до 200 символов)

5. Данные профиля:
- Имя и фамилия (до 50 символов каждое)
- Текущий пароль и новый пароль для смены

Выходные данные включают:

1. Ответы API в формате JSON:
- Расписание занятий с полями: id, subject, teacherId, startTime, endTime, room, floor, hasHomework
- Список дедлайнов с указанием статуса (PENDING, COMPLETED, OVERDUE) и временной метки completedAt
- Агрегированные рейтинги преподавателей (для обычных пользователей — без идентификаторов, для администраторов — с полной информацией)
- Stories с полями: id, title, description, imageUrl, linkUrl, linkText, viewedBy

2. Данные для отображения в интерфейсе:
- Форматированное расписание с группировкой по дням недели
- Визуализация сроков выполнения задач с цветовой индикацией: красный — сегодня, оранжевый — завтра, синий — позже
- Графическое представление рейтингов (звёзды от 1 до 5, средний балл)
- Stories с таймером автоматического переключения и индикатором прогресса (progress bar)

3. Локально сохранённые данные в shared_preferences:
- JWT токен аутентификации для автоматического входа (срок действия 7 дней)
- Настройки интерфейса: тема оформления (светлая или тёмная), язык
- Важно: расписание, дедлайны и другие бизнес-данные НЕ сохраняются локально

Обмен данными между клиентом и сервером осуществляется через REST API с использованием формата JSON. Используется протокол HTTP. JWT токены передаются в заголовке Authorization: Bearer для защиты endpoints. Локальное хранилище на устройстве реализовано через библиотеку shared_preferences только для токенов и настроек интерфейса, без кэширования бизнес-данных.

5. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

5.1. Ориентировочная экономическая эффективность

Разработка мобильного приложения Мобильное приложение для Гимназии им. Примакова для Гимназии им. Примакова направлена на повышение эффективности учебного процесса за счет автоматизации рутинных операций и централизации информации. Проект разработан в рамках курсовой работы и демонстрирует потенциальный экономический эффект при внедрении:

- Сокращение временных затрат на получение расписания и новостей гимназии
- Уменьшение нагрузки на административный персонал для информирования об изменениях в расписании или других аспектах учебного процесса
- Повышение вовлеченности за счет удобного интерфейса и персонализации данных

5.2. Предполагаемая потребность

Приложение разработано для учащихся, преподавательского состава и обслуживающего персонала Гимназии им. Примакова в рамках курсового проекта.

Основные сценарии использования:

1. Ежедневное получение актуального расписания занятий
2. Отслеживание дедлайнов и домашних заданий
3. Просмотр школьных новостей и анонсов через Stories с фотографиями и гиперссылками
4. Оценивание преподавателей по шкале от 1 до 5
5. Редактирование профиля пользователя (имя, фамилия, смена пароля)
6. Просмотр меню столовой с информацией о калорийности и аллергенах
7. Навигация по кампусу и поиск аудиторий
8. Создание заявок для сообщения о проблемах (IT, техническое обслуживание, уборка)
9. Просмотр объявлений гимназии

Приложение удовлетворяет потребность в удобном и быстром доступе к учебной информации, что особенно актуально в условиях интенсивного учебного процесса.

5.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами

Мобильное приложение для Гимназии им. Примакова сочетает функциональность нескольких категорий образовательных приложений, предлагая специализированное решение для конкретной гимназии. Ключевые отличия:

- Локализованный функционал, учитывающий специфику учебного процесса Гимназии им. Примакова
- Отсутствие необходимости адаптировать общие решения под конкретные нужды
- Расширение функционала существующей экосистемы гимназии
- Снижение нагрузки на учителей и сотрудников гимназии

Характеристика	PrimakovApp	МЭШ	Дневник.ру	Сетевой город
Персонализация под организацию	Полная	Частичная	Частичная	Частичная
Офлайн-режим	Нет (всегда актуальные данные)	Частичный	Частичный	Частичный
Интеграция с сайтом школы	Есть	Отсутствует	Отсутствует	Отсутствует
Система рейтингов преподавателей	Есть	Отсутствует	Отсутствует	Отсутствует
Stories с фотографиями	Есть	Отсутствует	Отсутствует	Отсутствует
Меню столовой с БЖУ и аллергенами	Есть	Отсутствует	Отсутствует	Отсутствует
Система заявок (IT/уборка/ТО)	Есть	Отсутствует	Отсутствует	Отсутствует
Стоимость развертывания	Низкая	Высокая	Средняя	Средняя

Таблица 1. Сравнение с аналогами

Как видно из таблицы, Мобильное приложение для Гимназии им. Примакова предлагает уникальное сочетание возможностей, специально разработанных под процессы гимназии. Отсутствие офлайн-режима является осознанным архитектурным решением: приложение всегда отображает актуальные расписание и объявления, что критично для образовательного процесса. Локальное хранение ограничено токенами доступа и настройками интерфейса, что повышает безопасность данных и исключает рассинхронизацию информации.

Специализированные модули (меню столовой с детальной информацией о блюдах, система заявок) реализованы под локальные сценарии использования и отсутствуют в массовых платформах. Единый пользовательский интерфейс разработан с учётом фирменного стиля гимназии и обеспечивает согласованный опыт взаимодействия для всех участников образовательного процесса.

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. Гивакс Д. Д. Паттерны проектирования API. — М. : ДМК Пресс, 2021. — 384 с.
15. Фримен Э., Робсон Э., Бейтс Б., Сьерра К. Паттерны проектирования. — 3-е изд. — СПб. : Питер, 2020. — 656 с.
16. Чернышев С. А. Основы Flutter: разработка кроссплатформенных мобильных приложений. — М. : СОЛОН-Пресс, 2022. — 320 с.
17. Node.js. Официальная документация. — URL: https://nodejs.org/docs/ (дата обращения: 10.02.2026).
18. Express.js. Официальная документация. — URL: https://expressjs.com/ (дата обращения: 12.02.2026).
19. PostgreSQL. Документация. — URL: https://www.postgresql.org/docs/ (дата обращения: 29.04.2026).

20. Flutter. Документация. — URL: https://docs.flutter.dev/ (дата обращения: 25.02.2026).
21. Riverpod. Документация. — URL: https://riverpod.dev/ (дата обращения: 03.03.2026).
22. Dio package. Документация. — URL: https://pub.dev/packages/dio (дата обращения: 10.03.2026).
23. shared_preferences package. Документация. — URL: https://pub.dev/packages/shared_preferences (дата обращения: 15.03.2026).
24. url_launcher package. Документация. — URL: https://pub.dev/packages/url_launcher (дата обращения: 20.03.2026).
25. TypeScript. Официальная документация. — URL: https://www.typescriptlang.org/docs/ (дата обращения: 25.03.2026).
26. Jest. Документация. — URL: https://jestjs.io/ (дата обращения: 18.04.2026).
27. Docker. Документация. — URL: https://docs.docker.com/ (дата обращения: 05.02.2026).
28. GitHub. Документация. — URL: https://docs.github.com/ (дата обращения: 12.04.2026).
29. Visual Paradigm Online. — URL: https://online.visual-paradigm.com/ (дата обращения: 10.05.2026).

ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ

Приложение	Ссылка
МЭШ (Московская электронная школа)	https://uchebnik.mos.ru/
Дневник.ру	https://dnevnik.ru/
Сетевой город. Образование	https://netschool.ru/
Google Classroom	https://classroom.google.com/
Edmodo	https://www.edmodo.com/
Schoology	https://www.schoology.com/
Canvas LMS	https://www.instructure.com/canvas

Дата обращения: 07.05.2026.

ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ

Термин	**Определение**
Clean Architecture	Архитектурный подход с четким разделением на слои (Domain, Application, Infrastructure, Presentation), обеспечивающий независимость бизнес-логики от внешних фреймворков и библиотек.
Domain-Driven Design (DDD)	Методология разработки программного обеспечения, ориентированная на предметную область, с использованием единого языка и выделением доменных сущностей.
Repository Pattern	Паттерн проектирования для абстракции доступа к данным, предоставляющий интерфейсы для работы с хранилищем без привязки к конкретной реализации (PostgreSQL, MongoDB и др.).
Result Pattern	Паттерн для явной обработки ошибок без использования исключений, возвращающий объект с полями isSuccess, isFailure, value и error.
Dependency Injection	Принцип инверсии зависимостей, при котором зависимости передаются через конструкторы классов, что упрощает тестирование и замену реализаций.
Value Object	Неизменяемый объект со встроенной валидацией, представляющий концепцию предметной области.
Use Case	Отдельная бизнес-операция в Application Layer, инкапсулирующая логику выполнения одного сценария использования системы.
Entity	Бизнес-сущность с уникальным идентификатором и поведением, представляющая ключевой объект предметной области.
JWT (JSON Web Token)	Открытый стандарт для создания токенов доступа, содержащих JSON с утверждениями и подписанных криптографически.
RBAC (Role-Based Access Control)	Система разграничения доступа на основе ролей пользователей, где каждой роли назначен набор разрешений.
REST API	Архитектурный стиль для построения веб-сервисов, использующий HTTP методы (GET, POST, PUT, DELETE) и представление ресурсов в формате JSON или XML.
PostgreSQL	Объектно-реляционная система управления базами данных с открытым исходным кодом, поддерживающая транзакции, триггеры и сложные запросы.
Express.js	Минималистичный и гибкий веб-фреймворк для Node.js, предоставляющий набор функций для создания веб-приложений и API.
Flutter	Кроссплатформенный фреймворк от Google для разработки мобильных, веб и desktop приложений из единой кодовой базы на языке Dart.

Термин	Определение
Riverpod	Библиотека для управления состоянием во Flutter приложениях, обеспечивающая compile-time безопасность и реактивное обновление UI.
TypeScript	Типизированный язык программирования, являющийся надмножеством JavaScript, добавляющий статическую типизацию.
Dio	HTTP-клиент для Dart/Flutter с поддержкой интерсепторов, таймаутов, отмены запросов и обработки ошибок.
Backend	Серверная часть приложения, отвечающая за бизнес-логику, обработку данных, аутентификацию и взаимодействие с базой данных.
Frontend	Клиентская часть приложения, отвечающая за пользовательский интерфейс и взаимодействие с пользователем.
API (Application Programming Interface)	Интерфейс для взаимодействия между программными компонентами, определяющий методы, форматы данных и правила обмена информацией.
Аутентификация	Процесс проверки подлинности пользователя путем сверки предоставленных учетных данных (email, пароль) с данными в системе.
Авторизация	Процесс проверки прав доступа аутентифицированного пользователя к определенным ресурсам или операциям системы.
Миграция базы данных	Управляемое изменение схемы базы данных с сохранением версионности и возможности отката, обеспечивающее синхронизацию структуры БД с кодом приложения.
State Management	Управление состоянием приложения, включающее хранение, обновление и распространение данных между компонентами пользовательского интерфейса.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					